

Morphic Studio Software Architecture Specification

Document 44 — Authentication, Authorization & Security Architecture

Version: 1.0

Status: Approved

1. Purpose

This document defines the authentication, authorization, and security architecture for Morphe Studio.

Its objective is to protect user identities, creative work, AI interactions, APIs, infrastructure, and future integrations while supporting long-term scalability and enterprise readiness.

2. Security Philosophy

Security follows five core principles:

2.1 Zero Trust

No request is automatically trusted.

Every request must be:

- Authenticated
 - Authorized
 - Validated
 - Logged
 - Audited
-

2.2 Least Privilege

Users, services, and AI components receive only the permissions required for their responsibilities.

Permissions should be granular and revocable.

2.3 Security by Default

Secure defaults are mandatory.

Examples include:

- HTTPS everywhere
 - Secure cookies
 - Strong password policies
 - CSRF protection
 - Content Security Policy
 - Input validation
-

2.4 Privacy First

Creators own their content.

Projects remain private unless explicitly shared.

User data must never be used for AI training without informed consent.

2.5 Defense in Depth

Multiple layers of protection are used.

If one control fails, others continue protecting the platform.

3. Identity Management

Support:

- Email/password authentication
- Secure password reset
- Email verification
- Remembered devices
- Session management

Future-ready support:

- Google
- GitHub
- Apple
- Microsoft
- Enterprise SSO
- Passkeys

Every user receives a permanent immutable User ID.

4. Session Management

Sessions should support:

- Multiple devices
- Session expiration
- Device naming
- Session revocation
- Refresh tokens
- Secure logout
- Idle timeout

Users can review and revoke active sessions.

5. Multi-Factor Authentication

Architecture should support:

- Authenticator apps
- Hardware security keys
- Email verification
- Recovery codes

MFA is optional for Version 1 but fully supported by the architecture.

6. Authorization Model

Implement Role-Based Access Control (RBAC).

Default roles:

- Owner
- Editor
- Viewer
- Administrator

Future roles:

- Team Admin
- Organization Admin
- Billing Manager
- Auditor

Permissions are assigned to roles rather than individual users.

7. Project Permissions

Projects support:

Owner

↓

Editors

↓

Commenters

↓

Viewers

Permissions may be inherited or overridden where appropriate.

8. API Security

Every API request must include:

Authentication

Authorization

Rate limiting

Input validation

Audit logging

Correlation ID

Sensitive operations require additional verification where appropriate.

9. AI Security

Every AI request is validated before execution.

The AI Gateway must enforce:

Prompt validation

Context filtering

Permission checks

Usage quotas

Audit logs

Provider isolation

User consent where required

10. Secret Management

Secrets must never be stored in source code.

Use environment variables or dedicated secret management solutions.

Examples:

API keys

Database credentials

Encryption keys

OAuth secrets

Webhook signing keys

Secrets should rotate regularly.

11. Encryption

Protect data:

In Transit

TLS 1.3 or newer

At Rest

Database encryption

Object storage encryption

Encrypted backups

Sensitive fields may receive additional application-level encryption.

12. Audit Logging

Audit events include:

Login

Logout

Permission changes

Project sharing

AI generation

Exports

Publishing

Administrative actions

Security events

Logs should be immutable and searchable.

13. Monitoring & Detection

Monitor:

Failed logins

Suspicious activity

API abuse

Rate-limit violations

Permission escalation

AI misuse

Unexpected system behavior

Critical events should trigger alerts.

14. Privacy Controls

Users should be able to:

View stored personal information

Manage connected accounts

Control AI usage preferences

Export account data

Delete their account (subject to retention policies)

Manage notification preferences

15. Compliance Readiness

Design the architecture to support future compliance efforts, including:

Data retention policies

Consent management

Audit trails

Data portability

Regional privacy requirements

The architecture should make future compliance achievable without major redesign.

16. Disaster Recovery

Support:

Credential recovery

Account recovery

Backup restoration

Incident response

Key rotation

Service recovery procedures

Security incidents should have documented response plans.

17. Acceptance Criteria

The security architecture is successful if:

- Every request is authenticated and authorized.
 - Permissions are enforced consistently.
 - Sensitive data is encrypted.
 - Secrets remain protected.
 - Audit logs provide traceability.
 - AI requests respect permissions.
 - Future enterprise security features integrate without redesign.
-

Conclusion

The Morphic Studio security architecture establishes a Zero Trust foundation that protects creators, projects, AI workflows, and platform infrastructure. By combining strong identity management, RBAC, encryption, auditing, privacy controls, and layered defenses, the platform is prepared for secure growth from Version 1 through future enterprise deployments.